

Sequences (CodeGrade)

<https://github.com/learn-co-curriculum/phase-3-sequences><https://github.com/learn-co-curriculum/phase-3-sequences/issues/new>

Learning Goals

- Utilize Python's `list`, `tuple`, `range`, and `str` data types to accomplish several common programming tasks.
- Execute and test Python code using the Python shell and `pytest`.

Key Vocab

- **Sequence:** a data structure in which data is stored and accessed in a specific order.
- **Index:** the location, represented by an integer, of an element in a sequence.
- **Iterable:** able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice:** a group of neighboring elements in a sequence.
- **Mutable:** an object that can be changed.
- **Immutable:** an object that cannot be changed. (*Many immutable objects appear mutable because programmers reuse their names for new objects.*)
- **List:** a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple:** an immutable data type in Python that can store many types of data.
- **Range:** a data type in Python that stores integers in a fixed pattern.
- **String:** an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Introduction

A *sequence* is a simple data structure that is present in all programming languages in which data is stored in a specified order. The elements of a sequence can be accessed by their *index*. Just as in JavaScript, indices of a sequence begin at 0 and increase by 1 with each step down the sequence:

```
// JavaScript
const my_list = [1, 2, 3, 4];
console.log(my_list[0]);
// 1
```

```
# Python
my_list = [1, 2, 3, 4]
print(my_list[0])
# => 1
```

`list` and `tuple` objects can store any type of data. `range` objects store only integers, and `str` objects can only store unicode characters.

When Are Sequences Used?

Whenever values are logically connected to one another and it is important to keep them in order, you should store them in a sequence data structure.

```
# A dynamic sequence of values that allows duplicates
fibonacci_list = [0, 1, 1, 2, 3, 5, 8, 13, 21]
```

```
# An immutable, ordered sequence of months
month_tuple = (
    'January',
    'February',
    'March',
    'April',
    'May',
    'June',
```

```
'July',  
'August',  
'September',  
'October',  
'November',  
'December',  
)  
  
# A simple pattern of numbers  
even_numbers_up_to_100 = range(0, 101, 2)  
  
# A grammatical sentence  
sentence_string = "Strings are immutable sequences of Unicode code points."
```

After being defined, sequences can be used as more complex data structures such as *queues* and *stacks*. We will discuss these more later on in this module.

► Which sequence type is surrounded by parentheses?

Common Sequence Operations

<https://docs.python.org/3/library/stdtypes.html#common-sequence-operations>

All sequence data types in Python support certain functions and operators and possess certain methods. For a sequence `s` :

- `x in s` returns `True` if `x` is equal to at least one element of `s` .
- `s + s2` returns a single sequence of the elements of `s` followed by the elements of `s2` .
- `s * n` returns a single sequence of `s` repeated `n` times.

- `s[i]` returns the `i` th element of `s` (starting at 0).

NOTE: Python also supports negative indices. `-1` represents the last element in a sequence.

- `s[i:j]` returns a *slice* of `s` from index `i` up to (but not including!) index `j`.
- `s[i:j:k]` returns a slice of `s` from `i` to `j` with steps of `k` in between.
- `len(s)` returns the number of elements in `s`.
- `min(s)` and `max(s)` return the minimum and maximum values in `s`, respectively. > NOTE: this requires the elements of `s` to be of the same data type, either `str` or numerical.
- `s.index(x)` returns the index of the first `x` in `s`.
- `s.count(x)` returns the number of instances of `x` in `s`.

Open up the Python shell and test each of these operations for yourself:

```
s = [4, 6, 3, 9, 3, 5, 1, 2]
1 in s
# => True
s + s
# => [4, 6, 3, 9, 3, 5, 1, 2, 4, 6, 3, 9, 3, 5, 1, 2]
s * 2
# => [4, 6, 3, 9, 3, 5, 1, 2, 4, 6, 3, 9, 3, 5, 1, 2]
s[1]
# => 6
s[-1]
# => 2
s[2:5]
# => [3, 9, 3]
s[2:5:2]
# => [3, 3]
len(s)
```

```
# => 8
min(s)
# => 1
max(s)
# => 9
s.index(3)
# => 2
s.count(9)
# => 1
```

► *How would you retrieve the last two elements of a sequence if you don't know the length?*

Lists

We've encountered lists in several of our lessons so far. As lists are mutable and can store any types of data in the same list object, they have many use cases and many operations, functions, and methods that allow us to manipulate their contents.

Sorting Lists

Python provides two straightforward solutions for sorting lists: the `list.sort()` method and the `sorted()` function. Both of these solutions require that all elements of the list be of the same data type.

`list.sort()` rearranges the elements of a list so that they are in order

```
# if we sort numbers, they will be sorted in ascending order
my_list = [3, 6, 4, 2, 1, 5]
my_list.sort()
print(my_list)
# [1, 2, 3, 4, 5, 6]
```

```
# If we sort strings, they will be sorted in alphanumeric order
my_list = ['Cabbage', 'Apple', 'Banana', 'Potato']
my_list.sort()
print(my_list)
#['Apple', 'Banana', 'Cabbage', 'Potato']
```

There are some parameters we can pass into `list.sort()` which can make the the function more versatile.

The `key` parameter allows us to pass in a function which can serve as a key for the sort comparison:

```
my_list = ['This is a long sentence', 'Word', 'z']

# What if we want to sort by the length of the string?
# We can use the key attribute to tell the sort function to sort using the len function.

my_list.sort(key = len)
print(my_list)
# => ['z', 'Word', 'This is a long sentence']

# If we want to sort in descending order we can pass in the reverse parameter into sort.
my_list = ['This is a long sentence', 'Word', 'z']
my_list.sort(key = len, reverse=True)
print(my_list)
# => ['This is a long sentence', 'Word', 'z']
```

We can sort complex data structures using sort `key`

Lets say we have a list of tuples we want to sort using the second key:

```
my_list = [('John', 2), ('Steve', 1), ('Joe', 3)]

# We can define a function for the list to sort by the second key
```

```
def sort_tuple(tuple_value):  
  
    # return the key we want to sort by  
    return tuple_value[1]  
  
my_list.sort(key = sort_tuple)  
print(my_list)  
# => [('Steve', 1), ('John', 2), ('Joe', 3)]
```

The `sorted()` function returns a sorted copy of the original list

This function should be used when you want to preserve the integrity of your original list, but you need a sorted version for a separate task.

```
my_list = [3, 6, 4, 2, 1, 5]  
sorted_list = sorted(my_list)  
print(my_list)  
# => [3, 6, 4, 2, 1, 5]  
print(sorted_list)  
# => [1, 2, 3, 4, 5, 6]
```

We can pass the `key` and `reverse` parameters into `sorted()` like we did for the `sort` method:

```
my_list = ['Loquacious', 'Chatty', 'Talkative']  
sorted_list = sorted(my_list, key=len, reverse=True)  
# => ['Loquacious', 'Talkative', 'Chatty']
```

Adding to Lists

Python allows us to modify any of the elements of a list using their index:

```
my_list = [0, 1, 2, 3]
my_list[0] = None
print(my_list)
# => [None, 1, 2, 3]
```

If we want to extend a list, we unfortunately cannot use the same approach:

```
my_list = [0, 1, 2, 3]
my_list[4] = 4
# => IndexError: list assignment index out of range
```

Python does provide us with two options to extend lists: `list.append()` and `list.insert()` .

`list.append()` does exactly what you might expect: it *appends* its parameter to the list.

```
my_list = [0, 1, 2, 3]
my_list.append(4)
print(my_list)
# => [0, 1, 2, 3, 4]
```

`list.insert()` provides us a few extra options for extending our list. Where `list.append()` can only add to the end of a list, `list.insert()` can insert at any index.

`list.insert()` takes two arguments: an index and a value. If a value already exists at the index, the new value is inserted before it and everything after is moved up by 1. If no value exists at the index, the new value is added to the end of the existing list.

```
my_list = ['a', 'b', 'c', 'd', 'f']
my_list.insert(4, 'e')
print(my_list)
# => ['a', 'b', 'c', 'd', 'e', 'f']
my_list.insert(1000, 'g')
```



```
print(my_list)
# => ['a', 'b', 'c', 'd', 'e', 'f', 'g']
```

► Which method allows you to insert characters into the middle of a list?

Removing from Lists

Python provides us with four (4!!!) options for removing elements from a list.

- The `del()` function.
- The `list.pop()` method.
- The `list.remove()` method.
- The `list.clear()` method.

`del()` removes elements from a list, specified by an index or range of indices.

```
my_list = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
del(my_list[0])
print(my_list)
# => ['b', 'c', 'd', 'e', 'f', 'g']
del(my_list[0:3])
print(my_list)
# => ['e', 'f', 'g']
```

`list.pop()` removes and returns the element at the index passed in as an argument. When used without any arguments, it removes and returns the last element of the list.

`list.remove()` removes the element passed in as an argument. This is one of the few `list` methods that searches by value instead of index!

`list.clear()` erases all of the values of a list. This is usually not a very useful tool, but it's a fast way to free up memory on your device if you're working with a particularly large list in the Python shell.

```
my_list = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
my_list.pop()
# => 'g'
my_list.pop(0)
# => 'a'
my_list.remove('f')
print(my_list)
# => ['b', 'c', 'd', 'e']
my_list.clear()
print(my_list)
# => []
```

Tuples

Tuples do not have any special exclusive methods like lists do. This is because tuples are *immutable*. The sequence that they are provided when they are created is maintained as long as the tuple object exists.

Ranges

Ranges are a very simple type of sequence that is most commonly used in **for** loops. Ranges can only contain integers in a fixed pattern. You can build a range using the **range()** constructor method and loop through it as if it were a standard list.

The range constructor only requires one argument: the end of the range. You may have noticed above that this top value is **not** included in the range itself. This is similar to how slicing works in lists.

```
for n in range(4):
    print(n)
```

```
# => 0
```

```
# => 1  
# => 2  
# => 3
```

There are two optional arguments that you can include when creating a range: a start value and a step size.

- `range(4)` gives us 0, 1, 2, 3
- `range(1, 4)` gives us 1, 2, 3
- `range(0, 4, 2)` gives us 0, 2

While ranges are very similar to lists, a range is a different data type and they exhibit some unique behaviors.

For instance, when we `print()` a list, we can see its contents:

```
my_list = [0, 1, 2, 3]  
print(my_list)  
# => [0, 1, 2, 3]
```

When we print a range containing the same elements, here's what we see:

```
my_range = range(4)  
print(my_range)  
# => range(0, 4)
```

► What would `range(2, 10, 2)` produce?

Strings

Python strings have *many* methods that allow you to access and manipulate their individual elements. It's very important to remember while working with strings that they are iterable objects (they can be looped through!) and they are indexed.

```
my_string = 'Hello world!'
for char in my_string:
    print(char)
```

```
# => H
# => e
# => l
# => l
# => o
# => 
# => w
# => o
# => r
# => l
# => d
# => !
```

```
my_string[0]
# => 'H'
```

Changing Case

Strings can be formatted for case using three methods:

- `str.upper()` returns an uppercase version of the original string.
- `str.lower()` returns a lowercase version of the original string.
- `str.title()` returns the original string in titlecase (with the first letter of each new word capitalized.)

[Other String Methods](https://www.w3schools.com/python/python_ref_string.asp) ➞ **[_\(https://www.w3schools.com/python/python_ref_string.asp\)](https://www.w3schools.com/python/python_ref_string.asp)**

There are many other string methods that will allow you to accomplish most simple reformatting and conversion operations that you'll want to do. There are so many, in fact, that we're providing a [link to a comprehensive list](https://www.w3schools.com/python/python_ref_string.asp)  (https://www.w3schools.com/python/python_ref_string.asp) instead of going in-depth on each one.

The important thing to remember when working with strings is that they are **immutable**. All string methods return a new object; they do not replace the original.

```
my_string = 'hello world!'
my_string.upper()
# => HELLO WORLD!
print(my_string)
# => hello world!
```

Instructions

Time to get some practice! Write your code in the `sequences.py` file in the `lib` folder. Run `pytest -x` to check your work. Your goal is to practice manipulating sequences with the Python tools you've learned about in this lesson and the lessons before.

Write a function `print_fibonacci()` that prints a list of the [fibonacci sequence](https://www.mathsisfun.com/numbers/fibonacci-sequence.html)  (<https://www.mathsisfun.com/numbers/fibonacci-sequence.html>) up to the length provided in the function's parameters.

```
print_fibonacci(9)
# => [0, 1, 1, 2, 3, 5, 8, 13, 21]
```

When all of your tests are passing, submit your work using `git`.

Resources

- [Common Sequence Operations](https://docs.python.org/3/library/stdtypes.html#common-sequence-operations)  (<https://docs.python.org/3/library/stdtypes.html#common-sequence-operations>)

- [Sequence Types](https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range)  [\(https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range\)](https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range)
- [More on Lists](https://docs.python.org/3/tutorial/datastructures.html#more-on-lists)  [\(https://docs.python.org/3/tutorial/datastructures.html#more-on-lists\)](https://docs.python.org/3/tutorial/datastructures.html#more-on-lists)
- [Range Docs](https://docs.python.org/3/library/stdtypes.html#ranges)  [\(https://docs.python.org/3/library/stdtypes.html#ranges\)](https://docs.python.org/3/library/stdtypes.html#ranges)
- [String Methods](https://www.w3schools.com/python/python_ref_string.asp)  [\(https://www.w3schools.com/python/python_ref_string.asp\)](https://www.w3schools.com/python/python_ref_string.asp)

This tool needs to be loaded in a new browser window

Load Sequences (CodeGrade) in a new window